Miscellaneous

Lecture 01: Vibe Coding

Prof. Dr. Michael Granitzer

Speaker notes

Diese Einheit sammelt erste strukturierte Gedanken zum Vibe Coding. Im Zentrum steht nicht die Frage, ob AI-Coding-Tools nuetzlich sind, sondern unter welchen Bedingungen sie produktiv werden. Die Arbeitsthese lautet: Vibe Coding funktioniert vor allem dort gut, wo starke Aufsicht, klare Ziele, gute Inspektion und systematische Verifikation vorhanden sind.

Ziel dieser Vorlesung

Vibe Coding ist keine magische Produktivitaetstechnik. Es ist eine Arbeitsweise, in der Menschen mit generativen Modellen Software schneller entwerfen, aendern und pruefen.

> Die zentrale Frage ist deshalb nicht: "Kann die AI coden?", sondern: **Unter welchen Arbeitsbedingungen wird daraus verlässliche Entwicklung?**

Leitfragen

- Warum funktioniert Vibe Coding fuer manche Teams erstaunlich gut, fuer andere aber kaum?
- Warum ist **Supervision** wichtiger als Prompt-Eleganz?
- Warum sollte man der AI eher das **Was** als das **Wie** vorgeben?
- Welche Inquiry- und Inspektionsstrategien machen die Arbeit belastbar?
- Welche Art von Tests ist im AI-unterstuetzten Coding besonders wichtig?

Speaker notes

Didaktisch lohnt sich eine Umkehr der ueblichen Perspektive. Viele Diskussionen ueber AI-Coding konzentrieren sich auf Modellfaehigkeiten. In der Praxis ist aber oft entscheidender, ob Menschen Anforderungen praezise formulieren, Ergebnisse kritisch pruefen und Fehler frueh sichtbar machen koennen.

Warum Vibe Coding funktioniert

Leitfrage: Welche Bedingungen machen AI-unterstuetztes Coding produktiv, statt nur schnell und gefaehrlich?

Speaker notes

Die erste Haelfte der Vorlesung formuliert eine positive These: Vibe Coding ist nicht per se schlecht, sondern in bestimmten Arbeitsumgebungen sogar sehr stark. Diese Bedingungen muessen explizit gemacht werden.

Was mit "Vibe Coding" gemeint ist

Typische Praxis

- Problem grob beschreiben
- Implementierungsideen erzeugen lassen
- Code direkt iterativ veraendern
- schnell testen, verwerfen, neu ansetzen
- viel ueber Dialog statt ueber Vorabplanung arbeiten

Typischer Reiz

- niedrige Einstiegshuerde
- hohe Geschwindigkeit bei Boilerplate und Umbauten
- schnelle Exploration unbekannter Libraries oder APIs
- gute Unterstuetzung fuer Prototyping und Glue Code

> Vibe Coding ist damit eher ein **interaktiver Arbeitsstil** als eine klar definierte Methode.

Speaker notes

Der Begriff ist unscharf und bewusst informell. Er bezeichnet meist eine Arbeitsweise mit hoher Dialogfrequenz und geringer Vorabplanung. Typisch ist, dass der Mensch Ideen, Zielbilder oder Probleme formuliert und das Modell konkrete Codevorschlaege, Umbauten oder Debugging-Hypothesen liefert.

Warum es fuer manche gut funktioniert

Bedingung	Warum hilfreich
Guter Problemrahmen	Die AI bekommt ein klares Zielbild
Schnelle Feedbackschleifen	Fehler werden frueh sichtbar
Gute Tooling-Umgebung	Diff, Tests, Logs und UI machen Ergebnisse pruefbar
Erfahrener Mensch in der Schleife	Schwache Stellen werden erkannt
Kleine bis mittlere Aufgaben	Komplexitaet bleibt lokal beherrschbar

Vibe Coding skaliert nicht automatisch mit Modellstaerke. Es skaliert oft besser mit **Sichtbarkeit, Kritikfaehigkeit und enger Rueckkopplung**.

Speaker notes

Viele Erfolgsgeschichten stammen aus Umgebungen, in denen die Aufgabe bereits gut verstanden ist und die Rueckmeldung schnell kommt. Wer lokal testen, im Browser inspizieren, Logs lesen und Aenderungen vergleichen kann, reduziert das Risiko erheblich. In solchen Umgebungen kompensiert das Arbeitssystem viele Schwaechen des Modells.

Warum es fuer andere scheitert

Problem	Typischer Effekt
Ziel unklar	Die AI produziert Aktivitaet statt Fortschritt
Mensch kann Ergebnis nicht beurteilen	Halluzinationen bleiben unentdeckt
Zu viel Vertrauen in fluessige Erklaerungen	Scheinplausibilitaet ersetzt Verifikation
Keine gute Inspektionsoberflaeche	Fehler bleiben unsichtbar
Kein Test- oder Integrationskontext	"Sieht gut aus" wird mit "funktioniert" verwechselt
Zu grosse Umbauten in einem Schritt	Lokale Fehler propagieren ins Gesamtsystem

> Der haeufigste Fehler ist nicht, dass die AI etwas Falsches produziert. Der haeufigste Fehler ist, dass Menschen **es zu spaet merken**.

Speaker notes

Vibe Coding scheitert besonders dort, wo Erkenntnis teuer ist. Wenn jemand weder die Domäne noch den Code noch die Laufzeitumgebung gut lesen kann, wird Geschwindigkeit zum Risiko. Fluent output erzeugt leicht die Illusion von Kontrolle, obwohl inhaltlich wenig abgesichert ist.

Operative Prinzipien

Leitfrage: Wie arbeitet man mit AI-Coding-Tools so, dass man Kontrolle behaelt?

Speaker notes

Hier geht es um konkrete Arbeitsprinzipien. Die Grundidee ist nicht, die AI moeglichst frei laufen zu lassen, sondern ihre Staerken in einen kontrollierten Engineering-Prozess einzubetten.

Supervision ist der Engpass

Die naive Sicht

- Hauptproblem sei Prompting
- bessere Formulierung loese fast alles
- Modellqualitaet bestimme den Erfolg

Die robustere Sicht

- Hauptproblem ist **Aufsicht**
- jemand muss Zwischenschritte beurteilen
- jemand muss falsche Abzweigungen frueh stoppen
- jemand muss Qualitaetskriterien durchsetzen

> Nicht Prompt Engineering ist die knappe Ressource, sondern **kompetente Supervision**.

Speaker notes

Supervision bedeutet hier mehr als Review am Ende. Gemeint ist laufende Kontrolle: Problem zuschneiden, Zwischenresultate pruefen, Annahmen hinterfragen, Scope reduzieren und schlechte Vorschlaege abbrechen. Ohne diese Aufsicht nimmt das Modell oft den plausibelsten statt den robustesten Weg.

Zielorientierung: Sage der AI was, nicht wie

Oft hilfreicher

- Zielzustand beschreiben
- Randbedingungen nennen
- Invarianten festlegen
- unerwuenschtes Verhalten explizit verbieten
- Erfolgskriterien benennen

Oft weniger hilfreich

- zu frueh die konkrete Implementierung diktieren
- interne Struktur ohne Not festlegen
- Modell auf einen einzigen Loesungspfad einsperren
- die Form der Loesung mit dem Zweck verwechseln

> Ein guter Prompt beschreibt haeufig zuerst **das Problem, die Grenzen und den Pruefrahmen** - nicht sofort die interne Mechanik.

Speaker notes

Das ist kein Freifahrtschein fuer unpraezise Aufgaben. Im Gegenteil: Zielorientierung verlangt sehr klare Anforderungen. "Was" meint hier Zielzustand, beobachtbares Verhalten, Constraints und Nicht-Ziele. "Nicht wie" bedeutet nur, dass man dem Modell die Ausgestaltung offen lassen kann, solange die Verifikation stark genug ist.

Inquiry-Strategien statt blinder Generierung

Strategie	Nutzen
Nach Annahmen fragen	Versteckte Unsicherheit wird sichtbar
Alternativen vergleichen lassen	Loesungsraum wird explizit
Zwischenschritte begruenden lassen	Denkfehler fallen frueher auf
Risiken und Failure Modes abfragen	Verifikation wird gezielter
Diffs statt Vollausgabe bevorzugen	Aenderungen bleiben lesbar
Vor dem Edit erst Analyse verlangen	Lokales Verstaendnis steigt

> Gute Zusammenarbeit mit AI ist oft weniger "schreib mir Code" und mehr **"zeige mir, wie du das Problem gerade modellierst"**.

Speaker notes

Inquiry bedeutet, das Modell nicht nur als Generator, sondern auch als Hypothesenpartner zu verwenden. Besonders wertvoll ist es, Annahmen, Alternativen und Risiken explizit zu machen. Dadurch wird die Arbeitsbeziehung weniger magisch und eher wie eine kontrollierte technische Diskussion.

Inspection UI entscheidet ueber Vertrauen

Was man sehen koennen sollte:

- Diff statt nur Endzustand
- Tests und Fehlermeldungen
- relevante Logs
- betroffene Dateien und Stellen
- Browser- oder API-Verhalten
- Build- und Laufzeitstatus

Wenn die Benutzeroberflaeche keine gute Inspektion erlaubt, wird Vertrauen schnell zu **Blindflug**.

Gerade deshalb funktionieren AI-Workflows oft besser in Umgebungen mit Terminal, Git-Diff, Testausgabe und DevTools als in reinen Chatfenstern.

Speaker notes

Inspektion ist ein infrastrukturelles Thema. Wer nur natuerlichsprachige Erklaerungen sieht, aber keine Diffs, Logs, Screenshots, Requests oder Tests, kann Fehler nur schwer systematisch erkennen. Eine gute Inspection UI senkt also nicht nur Friktion, sondern erhoeht epistemische Kontrolle.

Produktive Regel: Immer kritisch bleiben

> *Gute AI-Unterstuetzung reduziert Arbeit. Sie ersetzt nicht die Pflicht zum Zweifel.*

Praktische Konsequenzen

- nie Erklaerungen mit Korrektheit verwechseln
- nie "klingt sinnvoll" als Abnahmekriterium verwenden
- immer nach Gegenbeispielen suchen
- immer an Daten, Laufzeit und Integration denken
- immer pruefen, was sich durch eine Aenderung implizit mitveraendert

Speaker notes

Kritik ist kein Misstrauen aus Prinzip, sondern eine Form professioneller Qualitaetssicherung. Gerade weil die Modelle fluent und kooperativ wirken, muss der Mensch aktiv Gegenpruefungen organisieren. Sonst entsteht leicht ein Arbeitsstil, der subjektiv produktiv, aber objektiv fehleranfaellig ist.

Testen und Grenzen

Leitfrage: Welche Art von Verifikation passt besonders gut zu AI-unterstuetztem Coding?

Speaker notes

Der letzte Block fokussiert auf Verifikation. Wenn AI schnell Varianten produziert, muss das Testregime besonders gut darin sein, reale Integrationsprobleme sichtbar zu machen.

Testing: weniger nur Unit, mehr Integration

Warum Integrationstests wichtiger werden

- AI aendert oft mehrere Schichten gleichzeitig
- Fehler entstehen haeufig an Schnittstellen
- Konfigurations- und Datenflussprobleme sind oft zentral
- lokale Korrektheit garantiert noch kein Systemverhalten

Typische Pruefpunkte

- API-Endpunkte mit realistischen Requests
- Datenbank- und Persistenzpfade
- Frontend-Backend-Interaktion
- Auth-, Rechte- und Session-Pfade
- Build-, Start- und Deploybarkeit

Speaker notes

Unit-Tests bleiben nuetzlich, aber sie decken oft nur kleine Ausschnitte ab. AI-generierter Code scheitert in der Praxis haeufiger daran, dass Abhaengigkeiten, Schnittstellen oder Konfigurationen nicht sauber zusammenpassen. Deshalb sind Integrations- und Systemtests im AI-Kontext besonders wertvoll.

Synthetic Data Testing als Arbeitsmittel

Nutzen	Beispiel
Seltene Faelle gezielt erzeugen	Grenzwerte, Nullwerte, leere Antworten
Datenschutz wahren	keine produktiven Personendaten noetig
Testabdeckung verbreitern	viele Eingabevarianten systematisch pruefen
Failure Modes provozieren	kaputte Formate, schlechte Reihenfolgen, Konflikte

> Synthetische Daten sind nicht nur Ersatz fuer echte Daten. Sie sind ein Werkzeug, um **gezielt schwierige Faelle sichtbar zu machen**.

Speaker notes

Synthetic Data Testing passt gut zu AI-Workflows, weil sich damit schnell breite und kontrollierte Testsaetze erzeugen lassen. Besonders wertvoll ist das fuer Randfaelle, Privacy-sensitive Domänen und Fehlerbedingungen, die in echten Daten selten auftreten oder schwer reproduzierbar sind.

Wo Vibe Coding besonders riskant bleibt

Bereich	Warum riskant
Sicherheitskritische Logik	kleine Fehler haben grosse Wirkung
Verteilte Systeme	Fehler zeigen sich oft nur emergent
Legacy-Systeme	implizites Wissen fehlt im Prompt
Regulierte Domänen	Nachvollziehbarkeit und Compliance sind zentral
Grosse Refactorings	globale Seiteneffekte schwer ueberschaubar

> Je teurer Fehler werden und je spaeter sie sichtbar sind, desto weniger darf man sich auf "schnelle Plausibilitaet" verlassen.

Speaker notes

Nicht jede Aufgabe eignet sich gleich gut fuer exploratives AI-Coding. Besonders problematisch sind Domänen mit hohem Schadenspotenzial, starkem implizitem Kontext oder verteilten Seiteneffekten. Dort muss der Freiheitsgrad kleiner und die Verifikation dichter werden.

Arbeitsmodell: Vibe Coding unter Aufsicht

```
Zielbild und Constraints klaeren
        ->
AI fuer Analyse, Varianten und Entwurf nutzen
        ->
kleine Aenderung mit guter Sichtbarkeit erzeugen
        ->
Diff, Logs, UI und Verhalten inspizieren
        ->
Integration und Synthetic Data testen
        ->
erst dann uebernehmen oder verwerfen
```

> Vibe Coding wird dann belastbar, wenn es nicht als automatisches Coden verstanden wird, sondern als **schnelle Hypothesenerzeugung unter starker menschlicher Aufsicht**.

Speaker notes

Das Arbeitsmodell kombiniert Geschwindigkeit mit epistemischer Kontrolle. Die AI liefert Entwuerfe und Hypothesen, der Mensch liefert Problemrahmen, Kritik, Abbruchentscheidungen und Qualitaetsmassstaebe. Genau diese Kombination erklaert, warum dieselben Tools in unterschiedlichen Haenden sehr unterschiedliche Ergebnisse erzeugen.

Wrap-Up

- Vibe Coding funktioniert vor allem dort gut, wo **Supervision, Zielklarheit und schnelle Rueckmeldung** vorhanden sind.
- Der Mensch sollte der AI eher **das Was als das Wie** vorgeben: Ziele, Constraints, Nicht-Ziele und Erfolgskriterien.
- Inquiry-Strategien und gute Inspection UI sind entscheidend, weil sie Annahmen, Risiken und Fehler sichtbar machen.
- Kritisches Arbeiten bleibt Pflicht: fluessige Erklaerungen sind kein Beweis.
- Fuer Verifikation sind **Integrationstests und Synthetic Data Testing** besonders wichtig.

Speaker notes

Die Kernaussage ist bewusst nuanciert: Vibe Coding ist weder bloesse Spielerei noch automatische Produktivitaetsrevolution. Es ist eine nützliche, aber riskante Arbeitsform, deren Erfolg stark von menschlicher Aufsicht, guter Sichtbarkeit und systematischer Verifikation abhaengt.